```
arr = [fred, 10, 3.14, "This is a string", barney("pebbles"),]
```

Arrays of strings can be constructed using the shortcut notations %w and %W. The lowercase form extracts space-separated tokens into successive elements of the array. No substitution is performed on the individual strings. The uppercase version also converts the words to an array but performs all the normal double-quoted string substitutions on each individual word. A space between words can be escaped with a backslash. This is a form of general delimited input and is described on page 444.

```
arr = %w(fred wilma barney betty great\ gazoo)
arr       # => ["fred", "wilma", "barney", "betty", "great gazoo"]
arr = %w(Hey!\tIt is now -#{Time.now}-)
arr       # => ["Hey!\tIt", "is", "now", "-#{Time.now}-"]
arr = %W(Hey!\tIt is now -#{Time.now}-)
arr       # => ["Hey!   It", "is", "now", "-2023-01-17 00:08:01 -0600-"]
```

## Hashes

{*key expression* => *value expression*, ...}

{*key expression*: *value expression*, ...}

{*key and value expression*:}

A literal Ruby Hash is created by placing a list of key/value pairs between braces. Keys and values can be separated by the sequence =>. A comma appears between each key/value pair:

```
colors = {"red" => 0xf00, "green" => 0x0f0, "blue" => 0x00f}
```

If the keys are symbols, you can use this alternative notation:

```
colors = {red: 0xf00, green: 0x0f0, blue: 0x00f}
```

Strings will automatically convert to symbols if you use this notation, allowing for symbols with syntactically challenging names: {"do-this": 3, "do that": 4}, but note that the resulting keys will be symbols, not strings.

The keys and/or values in a particular hash need not have the same type. The keys and values will be accessible in sequence in the same order in which they were added to the hash.

If the value already exists in the context where the hash is being created, then you can leave off the value and the existing local value is used instead:

```
red = 0xf00
green = 0x0f0
blue = 0x00f
colors = {red:, green:, blue:}
```

### Requirements for a Hash Key

Hash keys must be objects that respond to the message hash by returning a hash code, and the hash code for a given key must not change. The keys used in hashes must also be comparable using eql?. If eql? returns true for two keys, then those keys must also have the same hash code. This means that certain classes (such as Array and Hash) can't conveniently be used as keys, because their hash values can change based on their contents.